

Metody Obliczeniowe w Nauce i Technice

Sprawozdanie z ćwiczenia 7

Rozwiązywanie układów równań liniowych metodami iteracyjnymi

Adrian Mikoda

27.05.2026 r.

Spis treści

1. Dane techniczne	1
2. Treści zadań	2
2.1. Zadanie nr 1	2
2.2. Zadanie nr 2	2
3. Realizacja ćwiczenia	3
4. Wyniki dla zadania 1	4
4.1. Kryterium Przyrostowe	4
4.1.1. Wektor początkowy $x_1^{(0)} = [0]^{n \times 1}$	4
4.1.2. Wektor początkowy $x_1^{(0)} = [50]^{n \times 1}$	5
4.1.3. Wektor początkowy $x_1^{(0)} = [100]^{n \times 1}$	7
4.2. Kryterium Rezydualne	8
4.2.1. Wektor początkowy $x_1^{(0)} = [0]^{n \times 1}$	8
4.2.2. Wektor początkowy $x_1^{(0)} = [50]^{n \times 1}$	10
4.2.3. Wektor początkowy $x_1^{(0)} = [100]^{n \times 1}$	11
5. Wyniki dla zadania 2	13
6. Wnioski	14
6.1. Zbieżność metody	14
6.2. Wpływ parametrów na proces iteracyjny	14
6.3. Dokładność rozwiązania	14
6.4. Porównanie kryteriów stopu	14

1. Dane techniczne

- System operacyjny: Windows 11 Home
- Procesor: AMD Ryzen 5 7530U (2.0 – 4.5 GHz)
- Pamięć RAM: 16GB (2133 MHz)
- Środowisko: Jupyter Notebook
- Obliczenia oraz wizualizacja: Python 3.13.7
- Użyte biblioteki: matplotlib 3.10.7 numpy 2.3.4 pandas 2.3.3
- Precyzja zmiennych: np.float32, np.float64

Dla powtarzalności losowań ustawiono `random.seed(42)`.

2. Treści zadań

Dany jest układ równań liniowych $Ax = b$.

Elementy macierzy A o wymiarze $n \times n$ są określone wzorem (m, k — parametry zadania podane indywidualnie):

$$\begin{cases} a_{i,i} = k \\ a_{i,j} = \frac{1}{|i-j|+m} \quad \text{dla } i \neq j \end{cases} \quad i, j = 1, \dots, n$$

Przyjmij wektor x jako dowolną n -elementową permutację ze zbioru $\{1, -1\}$ i oblicz wektor b . Potraktuj przyjęty wektor x jako x referencyjne.

Indywidualne parametry zadania: $k = 11, m = 3$.

2.1. Zadanie nr 1

Metodą Jacobiego rozwiąż układ równań liniowych $Ax = b$ (traktując jako niewiadomą wektor x), przyjmując kolejno kryterium stopu:

1. $\|x^{(i+1)} - x^{(i)}\| < \rho$
2. $\|Ax^{(i)} - b\| < \rho$

Obliczenia wykonaj dla różnych rozmiarów układu n , dla różnych wektorów początkowych, a także różnych wartości ρ w kryteriach stopu. (*Podaj, jak liczono normę.*) Wyznacz liczbę iteracji oraz sprawdź dokładność obliczeń (porównaj zgodnie z wybraną normą wektor x obliczony z układu z x referencyjnym). Sprawdź różnicę w czasie obliczeń dla obu kryteriów stopu.

Dodatkowe: Sprawdź wpływ x referencyjnego na obliczenia.

2.2. Zadanie nr 2

Dowolną metodą znajdź promień spektralny macierzy iteracji (dla różnych rozmiarów układu — takich, dla których znajdowane były rozwiązania układu). Sprawdź, czy spełnione są założenia o zbieżności metody dla zadanego układu. Opisz skrótowo przyjętą metodę znajdowania promienia spektralnego.

3. Realizacja ćwiczenia

Pierwszym krokiem związanym z realizacją zagadnienia było zaimplementowanie funkcji pomocniczych generujących macierz układu A oraz wektor prawych stron b , zgodnie z definicjami podanymi w treściach zadań.

Kolejnym etapem było zaimplementowanie metody Jacobiego z dwoma kryteriami stopu: przyrostowym oraz rezydualnym, a także funkcji pomocniczych do pomiaru czasu obliczeń, wyznaczania błędu maksymalnego oraz obliczania promienia spektralnego macierzy iteracji. Wyniki zebrano i zapisano do plików CSV, które następnie posłużyły do wygenerowania map ciepła.

Dla każdego rozmiaru układu n oraz każdego wektora początkowego $x^{(0)}$ wektor rozwiązania referencyjnego x przyjęto jako permutację ze zbioru $\{1, -1\}$ (z ustalonym ziarnem losowości), a następnie obliczono wektor $b = Ax$. Rozwiązując układ metodą Jacobiego, otrzymano wektor $\hat{x}$, który porównano z rozwiązaniem referencyjnym.

Błąd rozwiązania wyznaczono jako błąd maksymalny:

$$\max_i |x_i - \hat{x}_i|$$

Obliczenia przeprowadzono wyłącznie dla precyzji `float64`.

Macierz iteracji metody Jacobiego ma postać $M = -D^{-1}R$, gdzie D jest macierzą diagonalną macierzy A , a $R = A - D$. Ponieważ wszystkie elementy na przekątnej są równe k , odwrotność macierzy D sprowadza się do przemnożenia przez $1/k$.

Promień spektralny macierzy iteracji obliczono przy użyciu funkcji `numpy.linalg.eigvals`, wyznaczając maksymalną wartość bezwzględną spośród wszystkich wartości własnych.

Zbieżność metody Jacobiego jest zagwarantowana, gdy promień spektralny macierzy iteracji spełnia warunek $\rho(M) < 1$.

Eksperymenty przeprowadzono dla następujących wartości parametrów:

- rozmiary układu: $n \in \{3, 5, 10, 15, 20, 50, 100, 200, 500, 750, 1000\}$,
- wartości dokładności: $\rho \in \{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}, 10^{-7}, 10^{-10}, 10^{-15}\}$,
- wektory początkowe: $x^{(0)} = [0]^{n \times 1}$, $x^{(0)} = [50]^{n \times 1}$, $x^{(0)} = [100]^{n \times 1}$,
- kryteria stopu: przyrostowe oraz rezydualne,
- maksymalna liczba iteracji: 10000.

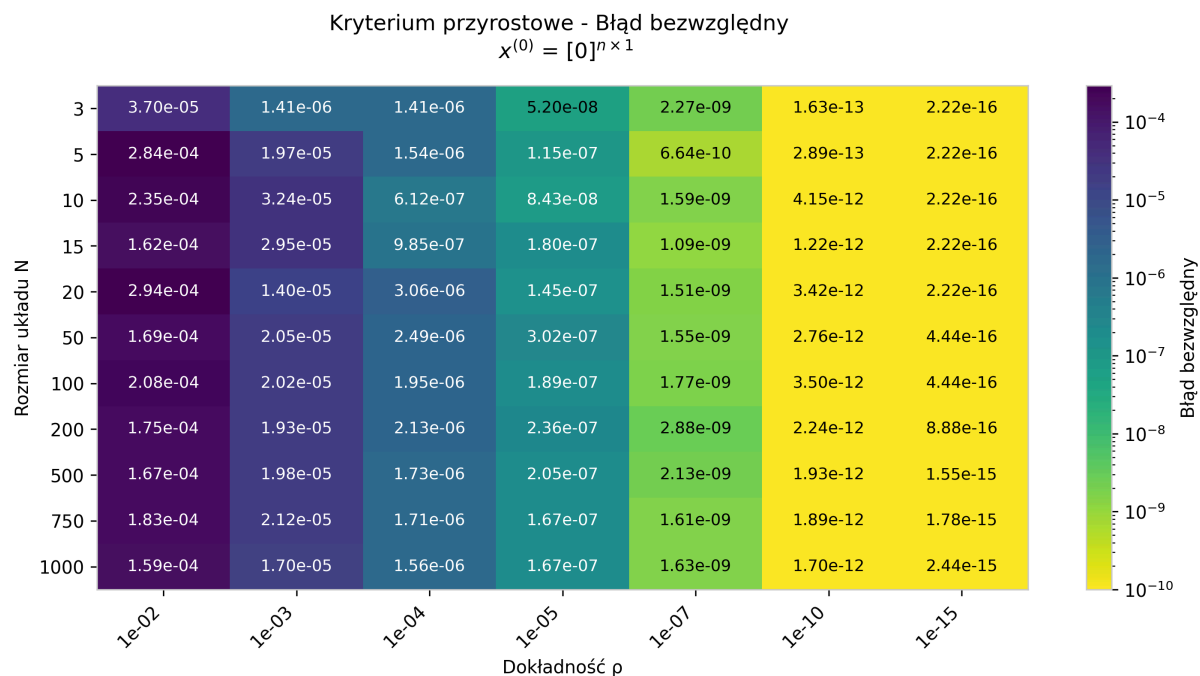
Czas obliczeń zmierzono przy użyciu funkcji `time.perf_counter` z biblioteki standardowej języka Python, mierząc czas bezpośrednio przed i po wywołaniu metody Jacobiego. Raportowany czas to średni czas na iterację.

Otrzymane wyniki przedstawiono w postaci map ciepła w dalszej części sprawozdania.

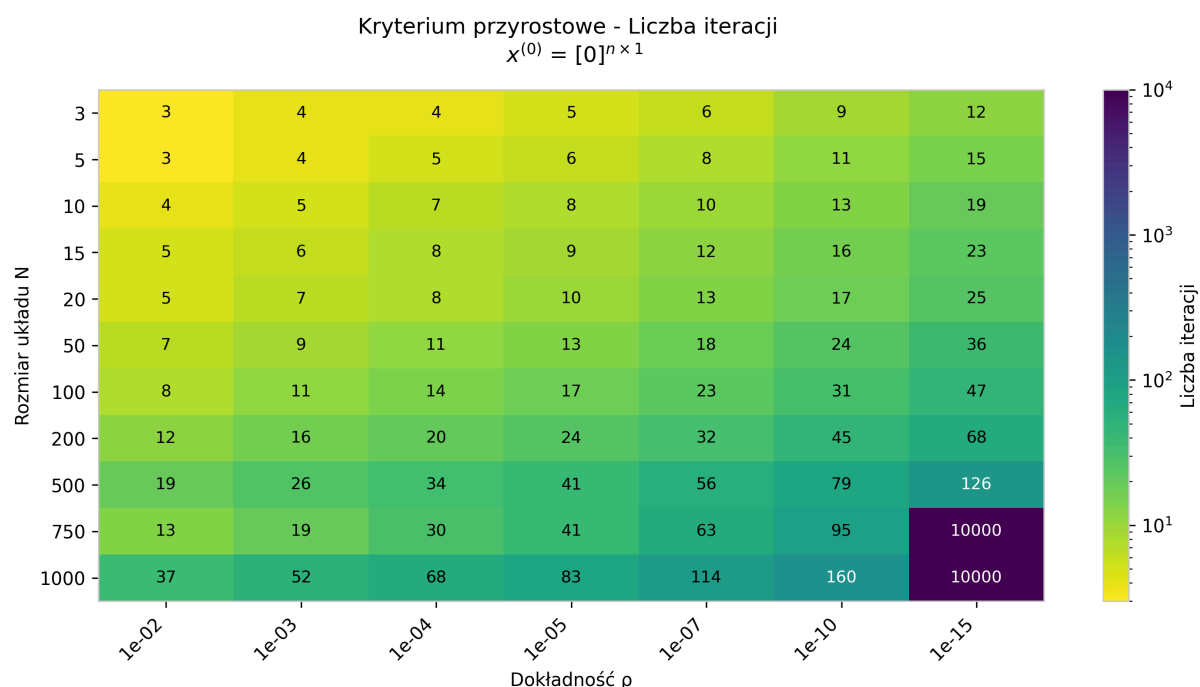
4. Wyniki dla zadania 1

4.1. Kryterium Przyrostowe

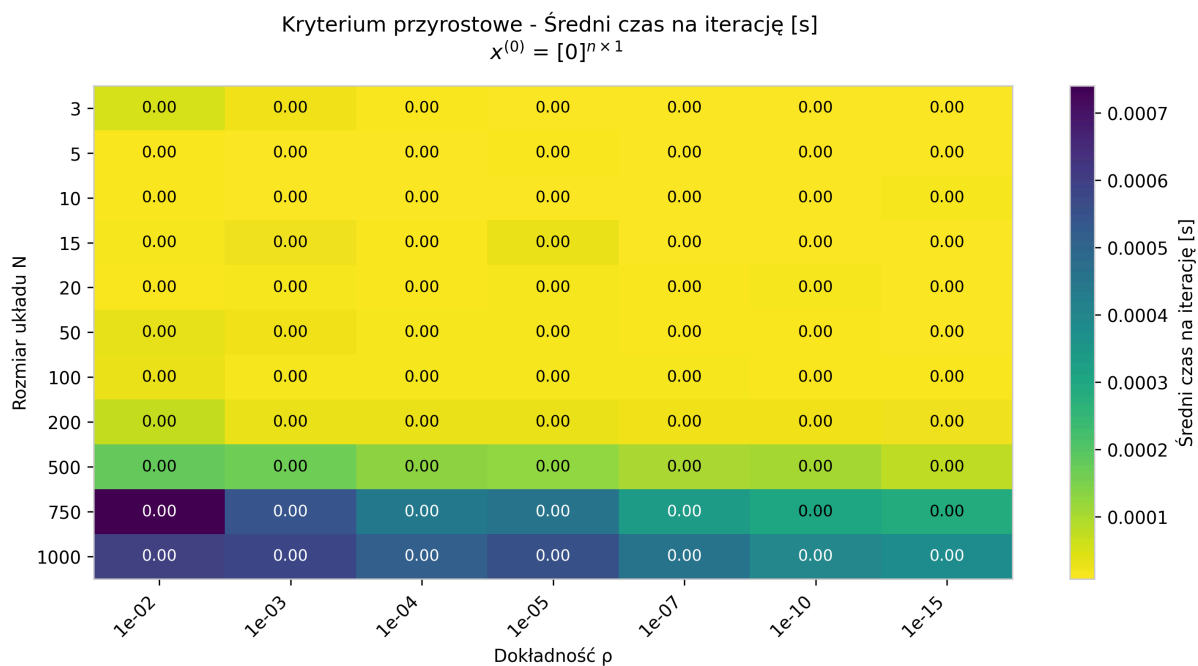
4.1.1. Wektor początkowy $x_1^{(0)} = [0]^{n \times 1}$



Rysunek 1: Błąd dla metody Jacobiego z kryterium przyrostowym oraz wektorem początkowym $x_1^{(0)}$

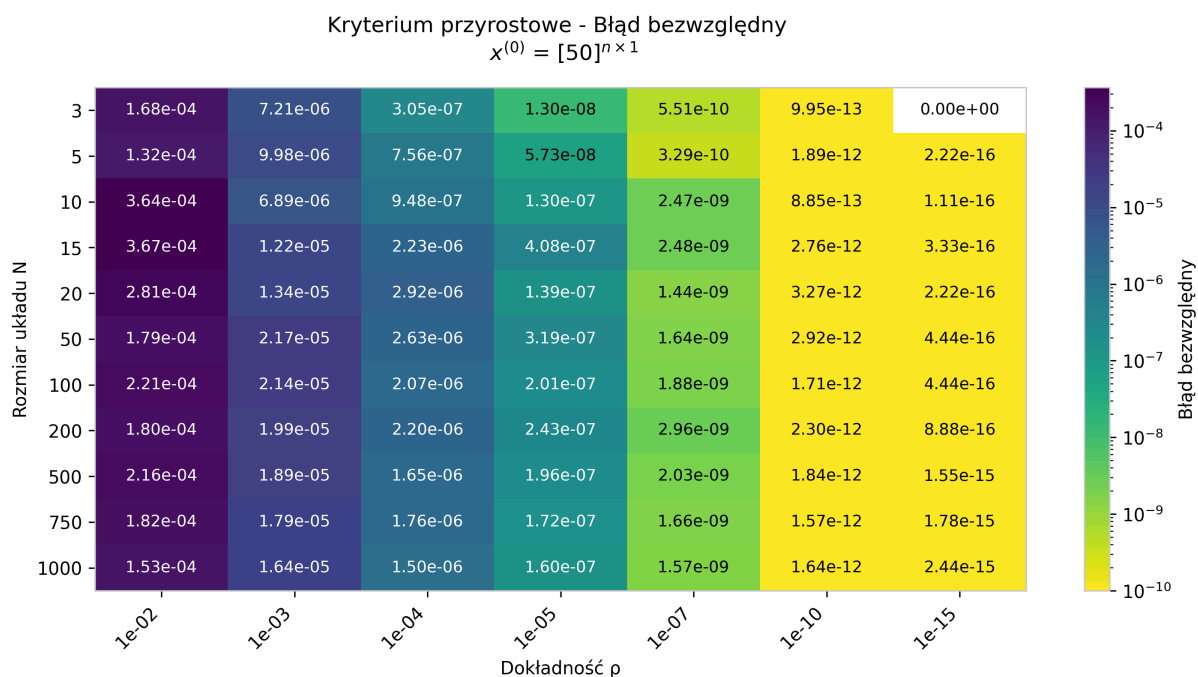


Rysunek 2: Liczba iteracji dla metody Jacobiego z kryterium przyrostowym oraz wektorem początkowym $x_1^{(0)}$

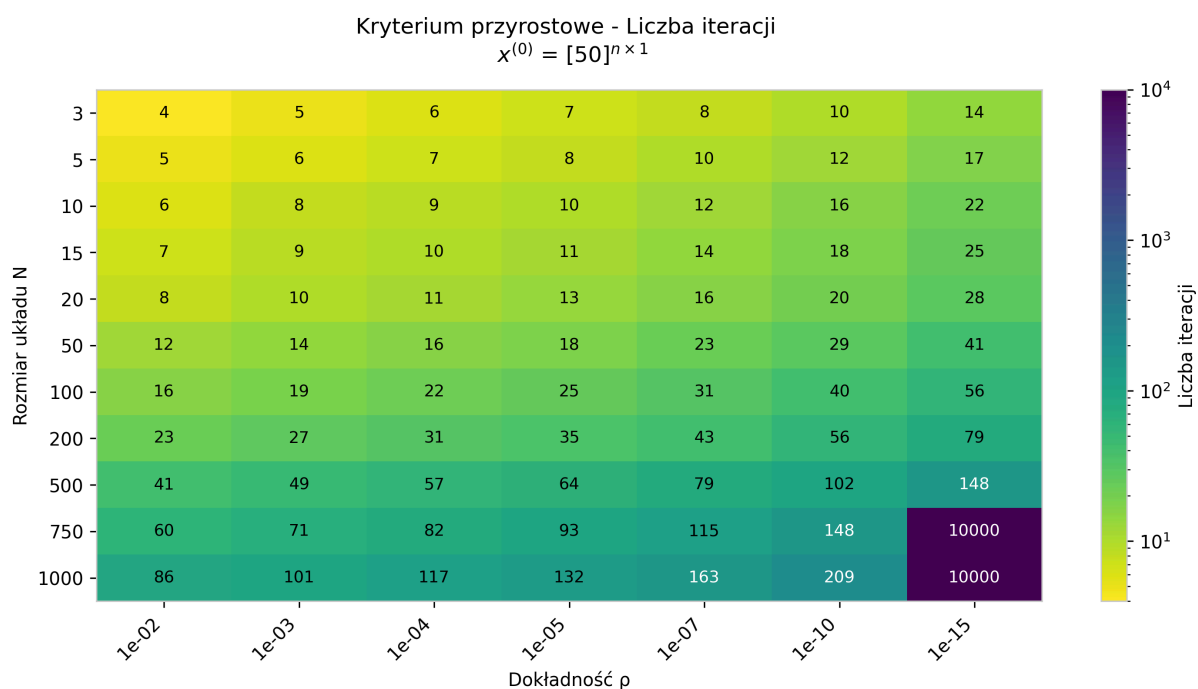


Rysunek 3: Średni czas obliczeń dla metody Jacobiego z kryterium przyrostowym oraz wektorem początkowym $x_1^{(0)}$

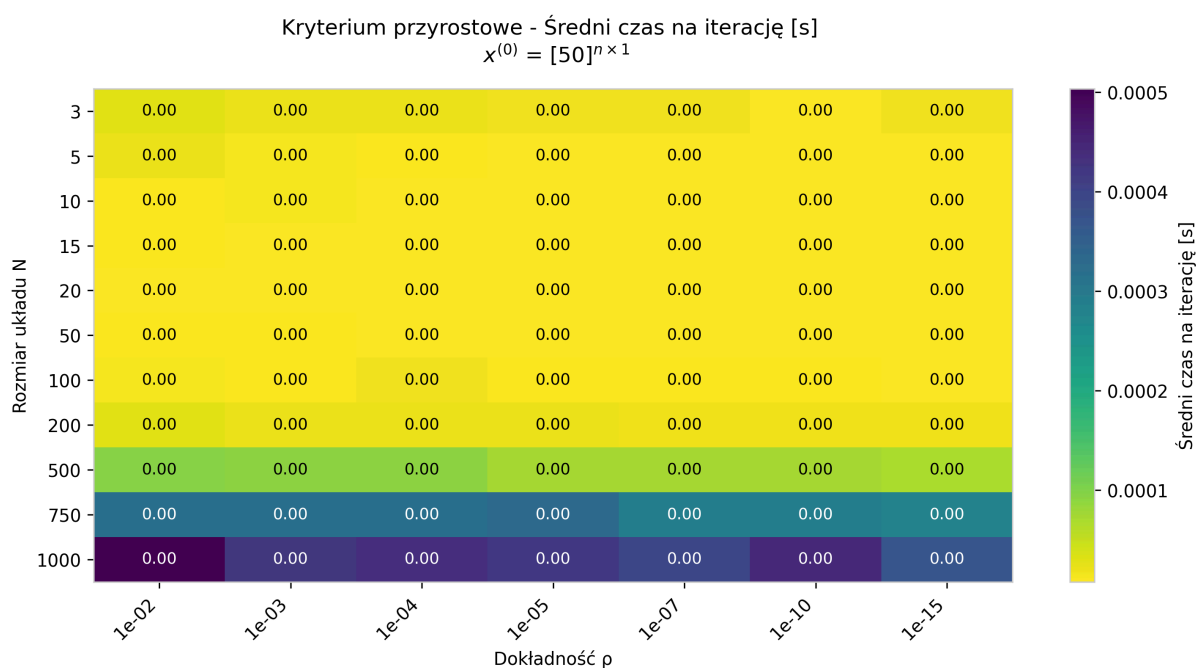
4.1.2. Wektor początkowy $x_1^{(0)} = [50]^{n \times 1}$



Rysunek 4: Błąd dla metody Jacobiego z kryterium przyrostowym oraz wektorem początkowym $x_1^{(0)} = [50]^{n \times 1}$

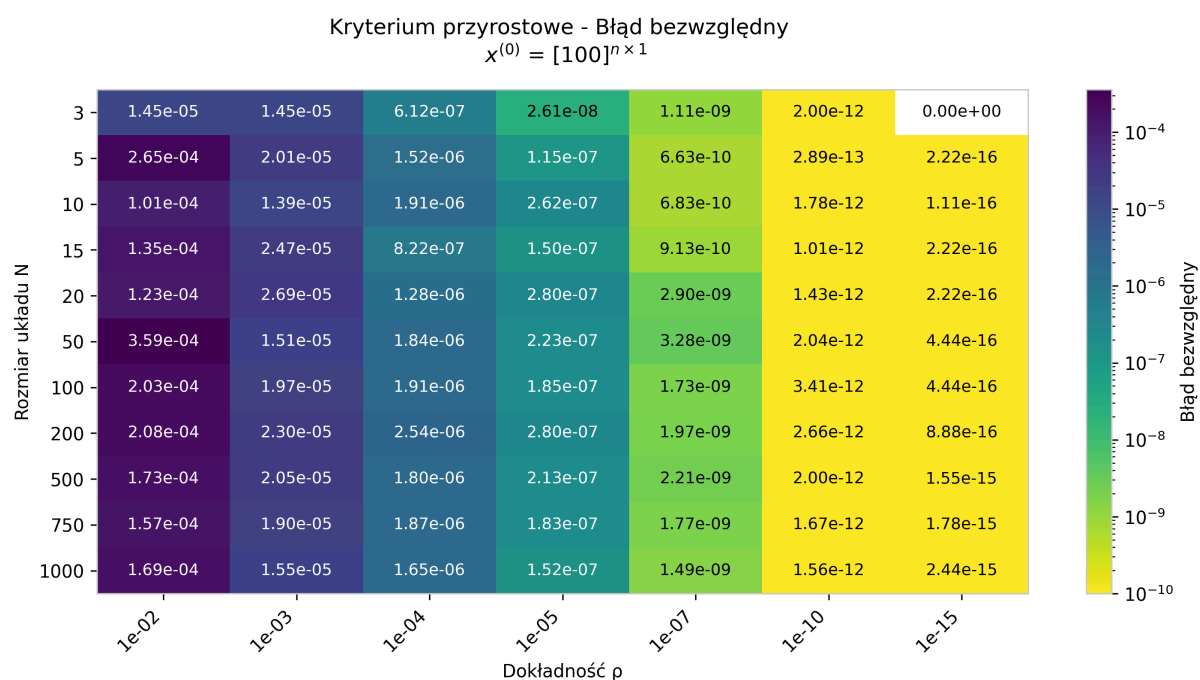


Rysunek 5: Liczba iteracji dla metody Jacobiego z kryterium przyrostowym oraz wektorem początkowym $x_1^{(0)} = [50]^{n \times 1}$

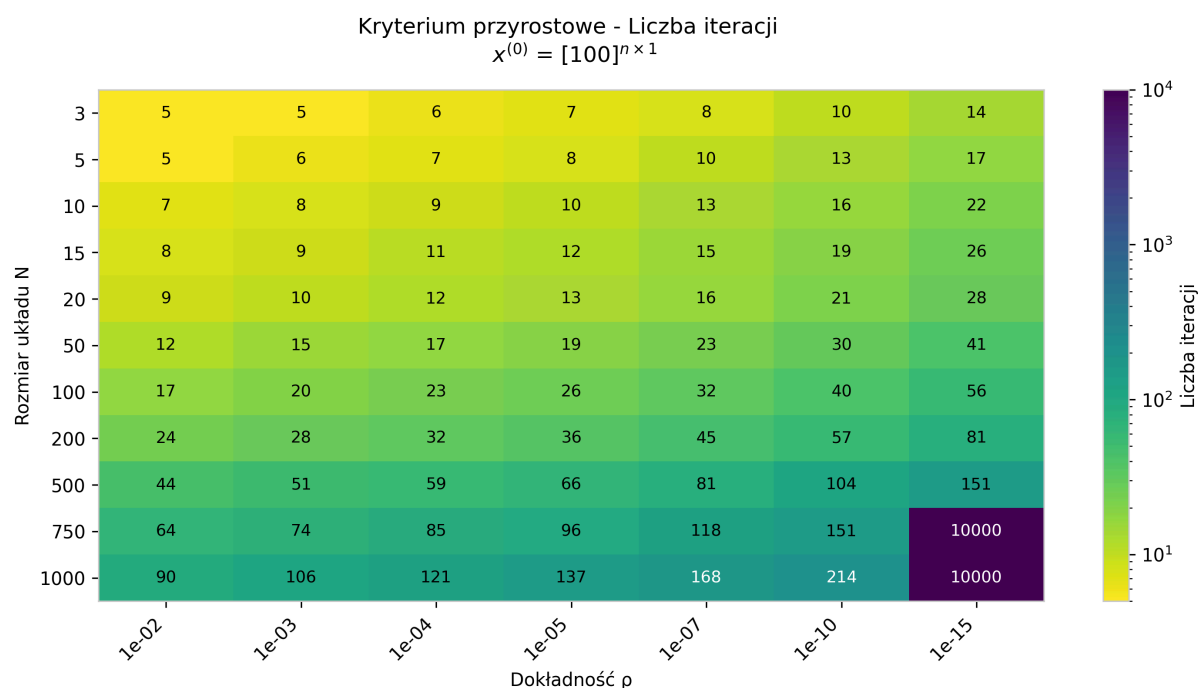


Rysunek 6: Średni czas obliczeń dla metody Jacobiego z kryterium przyrostowym oraz wektorem początkowym $x_1^{(0)} = [50]^{n \times 1}$

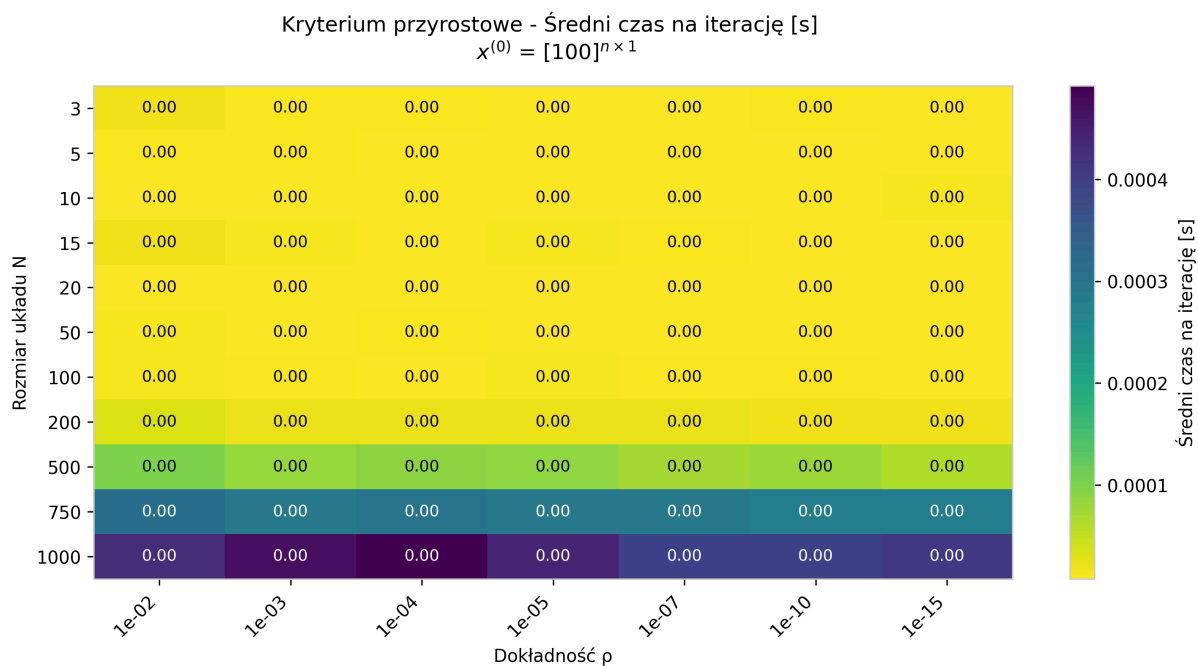
4.1.3. Wektor początkowy $x_1^{(0)} = [100]^{n \times 1}$



Rysunek 7: Błąd dla metody Jacobiego z kryterium przyrostowym oraz wektorem początkowym $x_1^{(0)} = [100]^{n \times 1}$



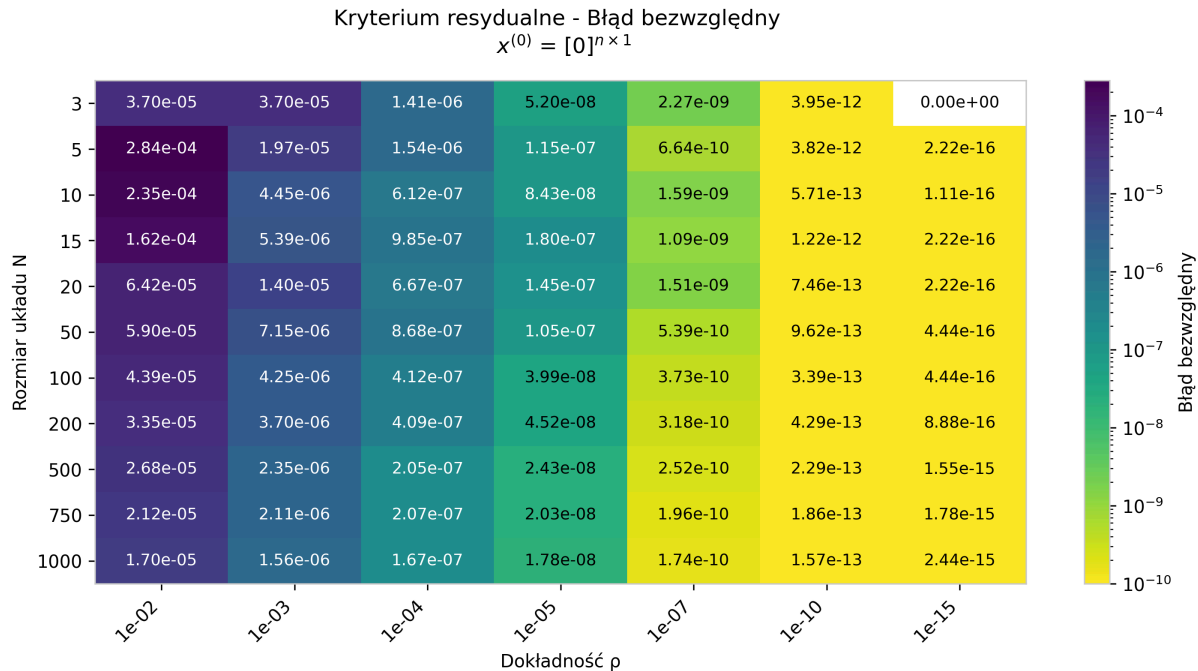
Rysunek 8: Liczba iteracji dla metody Jacobiego z kryterium przyrostowym oraz wektorem początkowym $x_1^{(0)} = [100]^{n \times 1}$



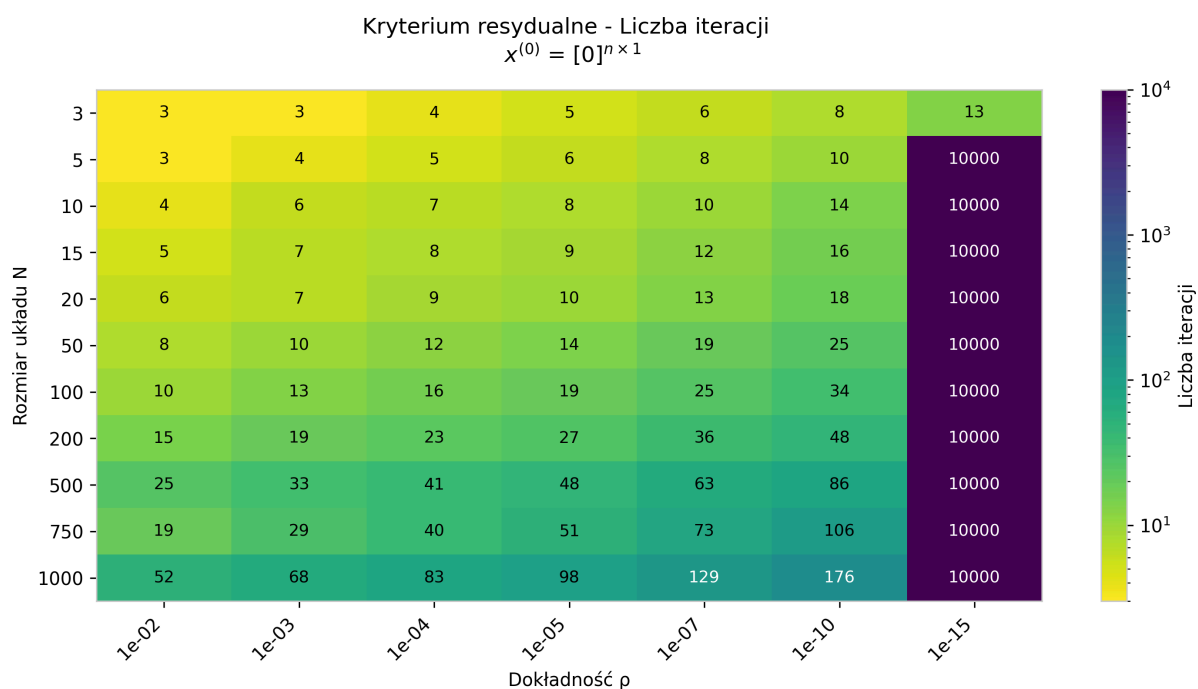
Rysunek 9: Średni czas obliczeń dla metody Jacobiego z kryterium przyrostowym oraz wektorem początkowym $x_1^{(0)} = [100]^{n \times 1}$

4.2. Kryterium Rezydualne

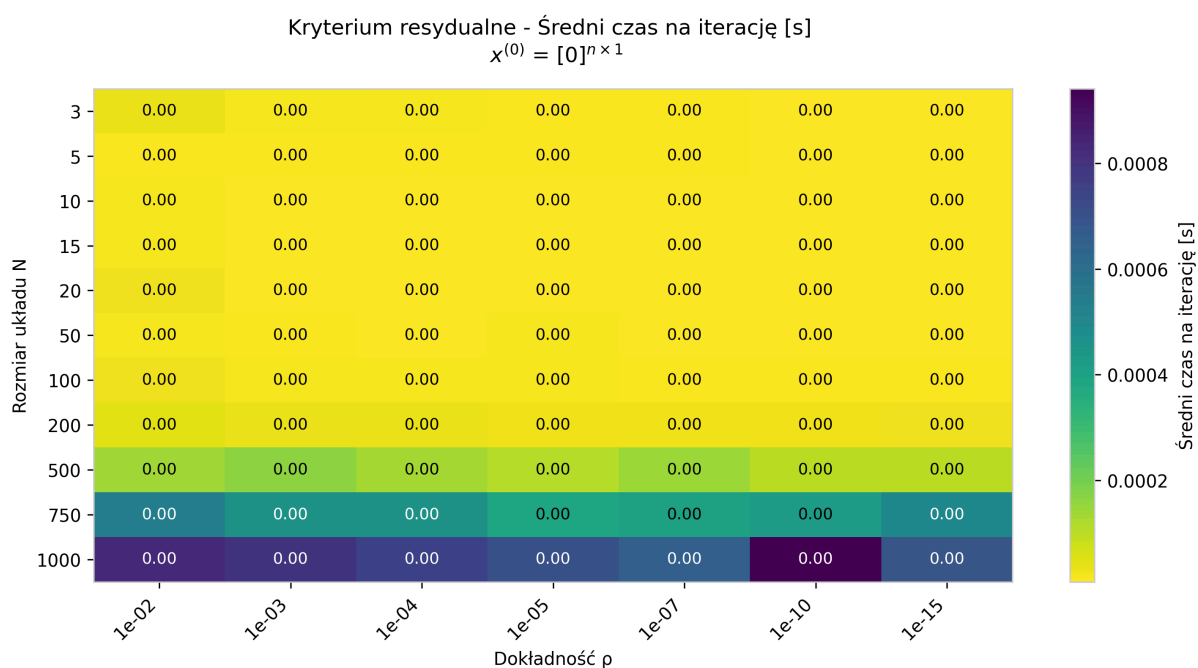
4.2.1. Wektor początkowy $x_1^{(0)} = [0]^{n \times 1}$



Rysunek 10: Błąd dla metody Jacobiego z kryterium rezydualnym oraz wektorem początkowym $x_1^{(0)}$

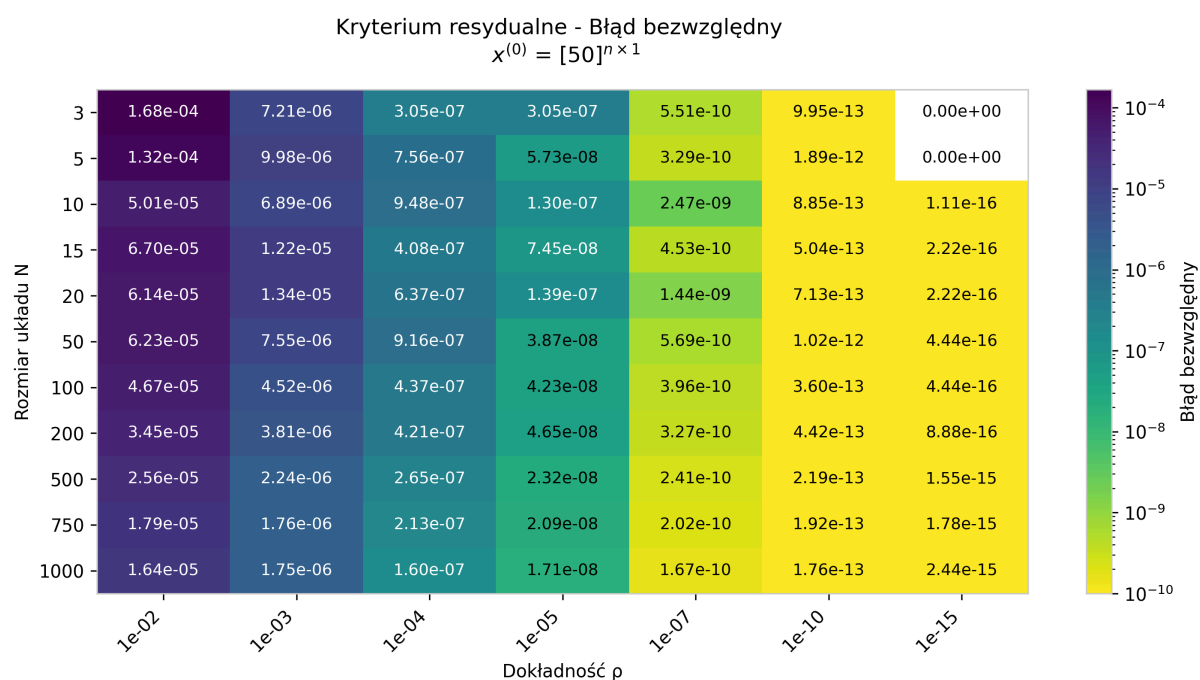


Rysunek 11: Liczba iteracji dla metody Jacobiego z kryterium rezydualnym oraz wektorem początkowym $x_1^{(0)}$

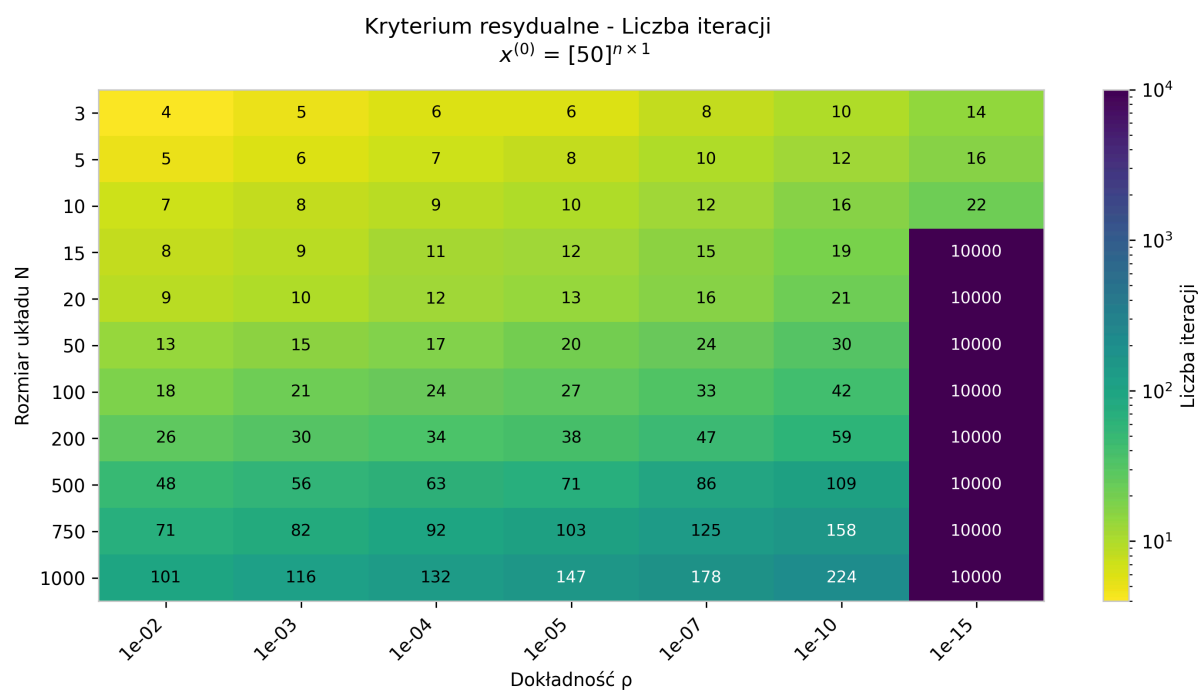


Rysunek 12: Średni czas obliczeń dla metody Jacobiego z kryterium rezydualnym oraz wektorem początkowym $x_1^{(0)}$

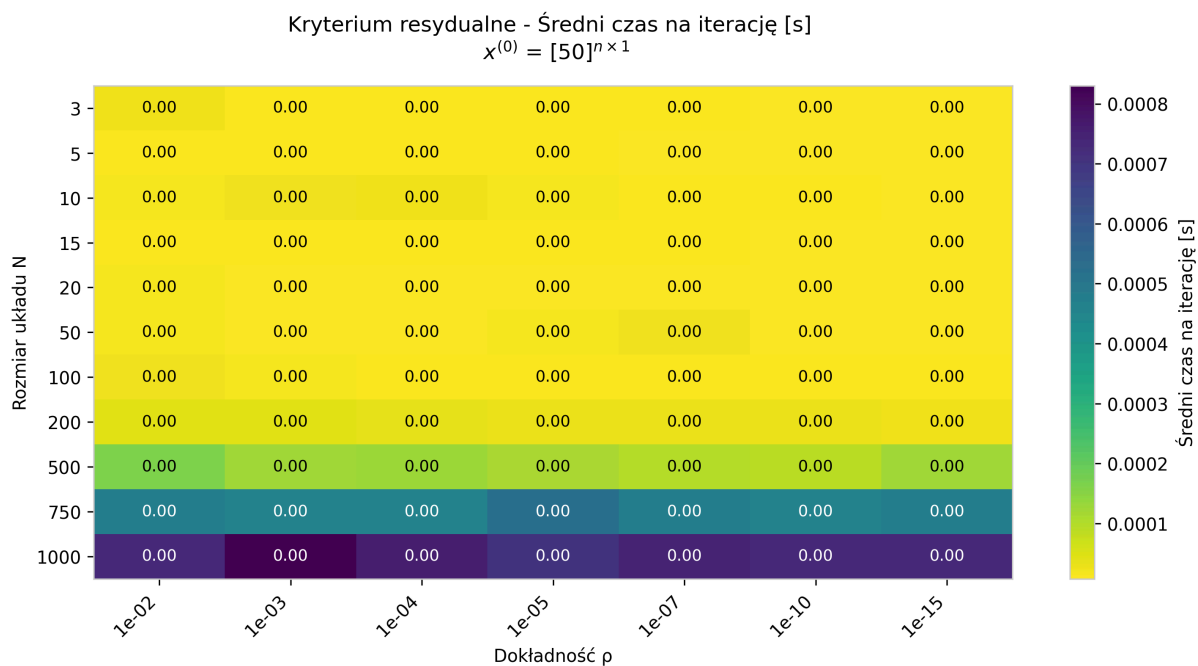
4.2.2. Wektor początkowy $x_1^{(0)} = [50]^{n \times 1}$



Rysunek 13: Błąd dla metody Jacobiego z kryterium rezydualnym oraz wektorem początkowym $x_1^{(0)} = [50]^{n \times 1}$

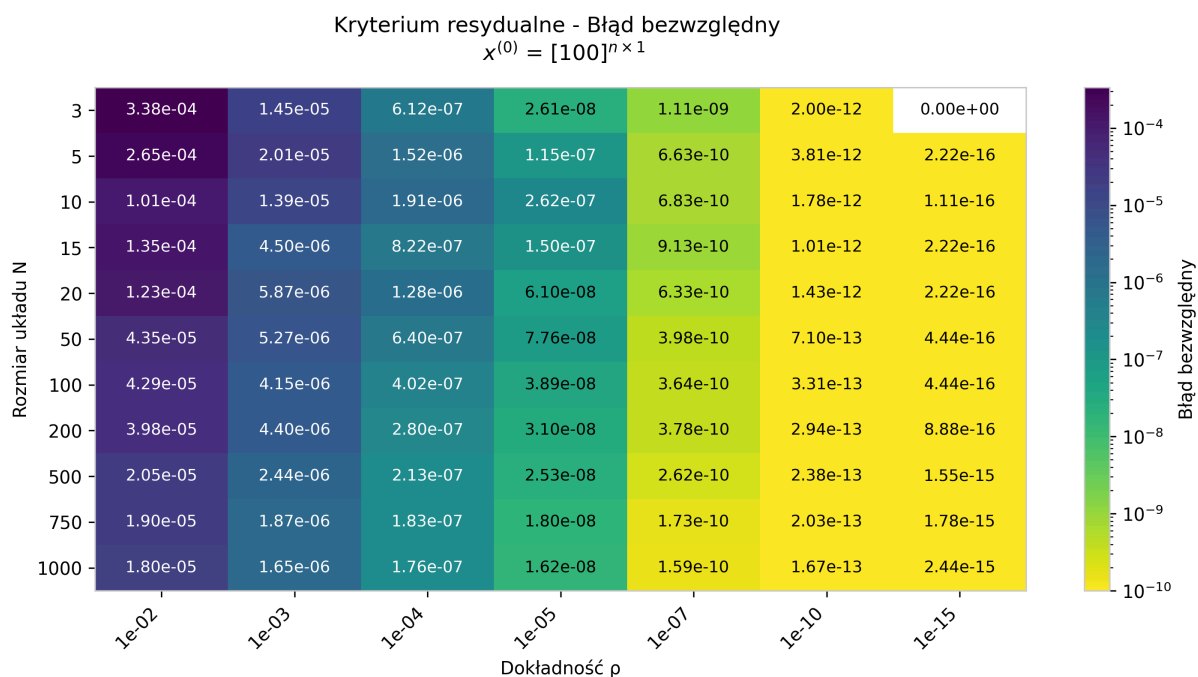


Rysunek 14: Liczba iteracji dla metody Jacobiego z kryterium rezydualnym oraz wektorem początkowym $x_1^{(0)} = [50]^{n \times 1}$

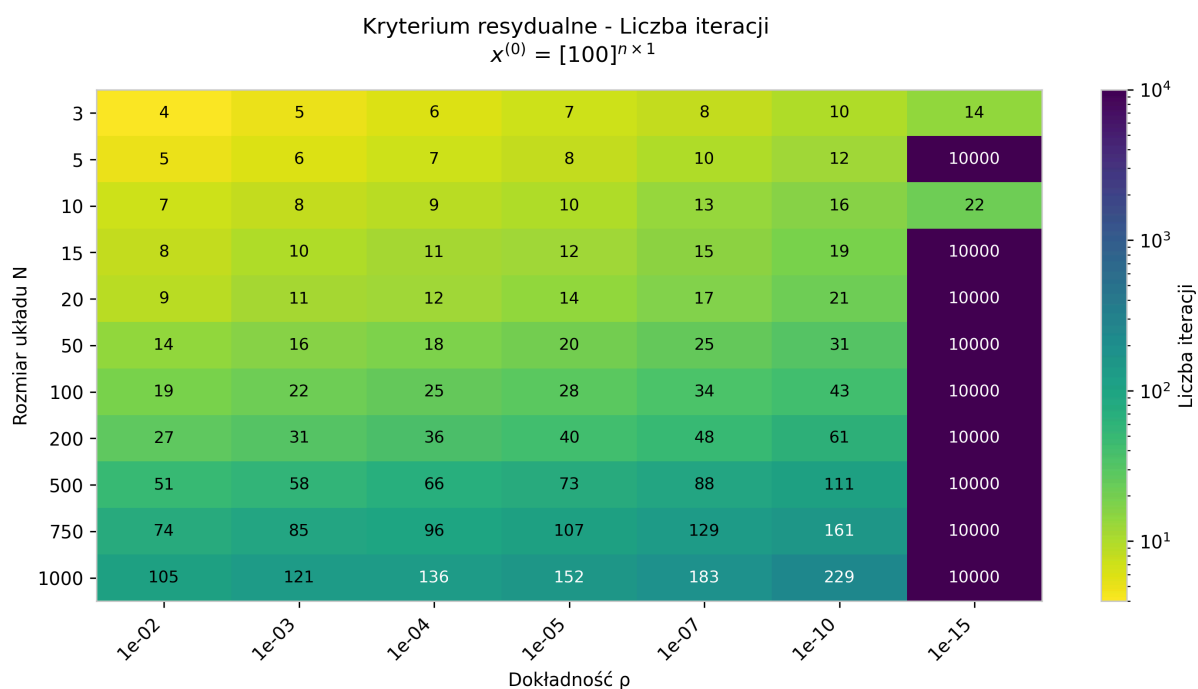


Rysunek 15: Średni czas obliczeń dla metody Jacobiego z kryterium rezydualnym oraz wektorem początkowym $x_1^{(0)} = [50]^{n \times 1}$

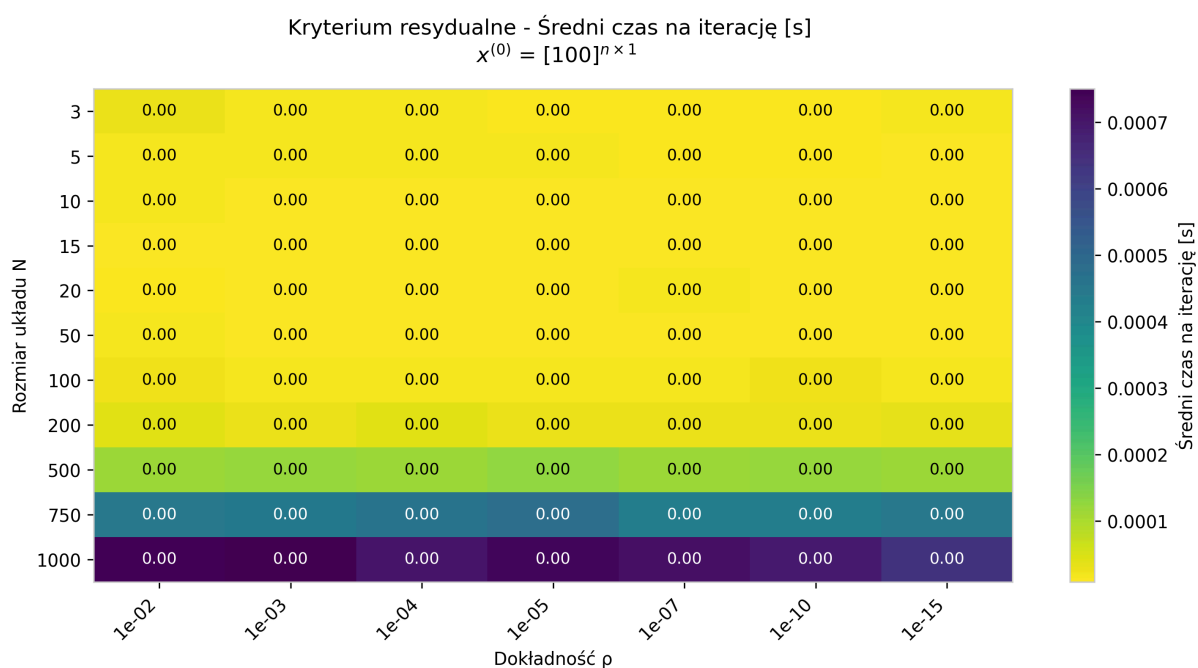
4.2.3. Wektor początkowy $x_1^{(0)} = [100]^{n \times 1}$



Rysunek 16: Błąd dla metody Jacobiego z kryterium rezydualnym oraz wektorem początkowym $x_1^{(0)} = [100]^{n \times 1}$

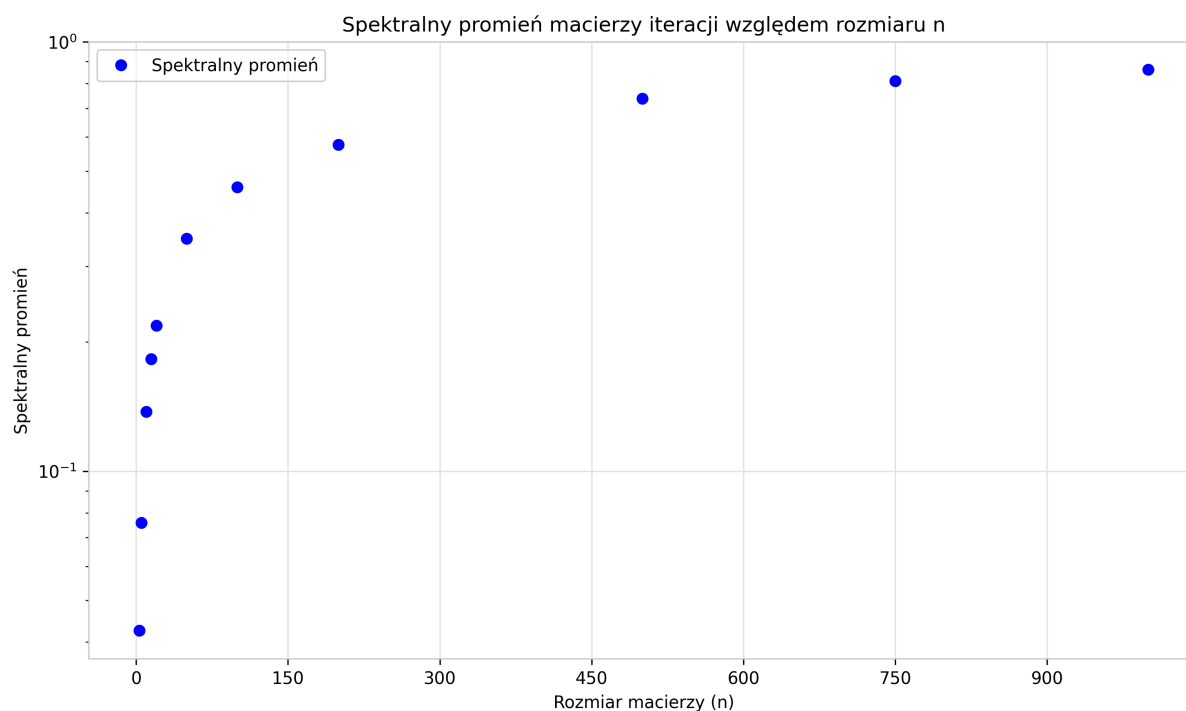


Rysunek 17: Liczba iteracji dla metody Jacobiego z kryterium rezydualnym oraz wektorem początkowym $x_1^{(0)} = [100]^{n \times 1}$



Rysunek 18: Średni czas obliczeń dla metody Jacobiego z kryterium rezydualnym oraz wektorem początkowym $x_1^{(0)} = [100]^{n \times 1}$

5. Wyniki dla zadania 2



Rysunek 19: Spektralny promień macierzy iteracji względem rozmiaru n

Promień spektralny macierzy iteracji pozostaje stały dla dużych n . Pozwala to na rozwiązywanie układów równań rzędu większego niż $n = 1000$.

6. Wnioski

6.1. Zbieżność metody

Analiza promienia spektralnego ρ_s macierzy iteracji $D^{-1}R$ wykazała, że dla wszystkich badanych rozmiarów macierzy n wartość $\rho_s < 1$, co potwierdza teoretyczny warunek zbieżności metody Jacobiego dla badanego układu. Promień spektralny rósł wraz ze wzrostem n , stabilizując się w okolicach 0.6065 dla większych macierzy.

6.2. Wpływ parametrów na proces iteracyjny

Rozmiar macierzy n : Wzrost n skutkował monotonicznym wzrostem liczby iteracji oraz średniego czasu pojedynczej iteracji — zgodnie z oczekiwaniami, gdyż każda iteracja wymaga operacji na wektorach rozmiaru n .

Próg zbieżności ρ : Zmniejszanie ρ prowadziło do wzrostu liczby iteracji w obu kryteriach. Dla $\rho = 10^{-15}$ kryterium przyrostowe osiągało limit 10000 iteracji jedynie dla największych układów ($n = 750, 1000$), natomiast kryterium rezydualne osiągało ten limit już od $n = 5$, co wskazuje na jego znacznie większą wrażliwość na ograniczenia precyzji maszynowej przy tak rygorystycznym progu.

Wektor początkowy $x^{(0)}$: Wektor zerowy $x^{(0)} = [0]^{n \times 1}$ wymagał wyraźnie mniejszej liczby iteracji niż wektory $[50]^{n \times 1}$ i $[100]^{n \times 1}$. Różnica między wektorami $[50]$ i $[100]$ była niewielka — oba wymagały zbliżonej liczby iteracji, nieznacznie rosnącej z odległością od rozwiązania.

6.3. Dokładność rozwiązania

Osiągany błąd bezwzględny malał wraz ze zmniejszaniem progu ρ dla obu kryteriów. Kryterium rezydualne dawało konsekwentnie niższe błędy niż przyrostowe przy tej samej wartości ρ — różnica wynosiła typowo jeden rząd wielkości. Dla najbardziej rygorystycznych progów błędy zbliżały się do poziomu epsilon maszynowego (rzędu $10^{-15} - 10^{-16}$).

6.4. Porównanie kryteriów stopu

Kryterium rezydualne zapewnia lepszą kontrolę nad faktycznym błędem rozwiązania, osiągając niższe błędy bezwzględne. Jednak przy $\rho = 10^{-15}$ jest praktycznie niezdolne do zbieżności dla większości rozmiarów układu, osiągając limit iteracji.